Davante

13

ENTORNOS DE DESARROLLO

Pruebas en el proceso de desarrollo software

ÍNDICE

/ 1. Introducción y contextualización práctica 3

/ 2. Planificación de las pruebas 4

/ 3. Tipos de pruebas 4

/ 4. Caso práctico 1: “Comprobación de funcionalidades” 5

/ 5. Casos de prueba 5

/ 6. Procedimientos en realización de pruebas 6

/ 7. Caso práctico 2: “Elección adecuada del tipo de prueba” 7

/ 8. Resumen y resolución del caso práctico de la unidad 7

/ 9. Bibliografía 8

© MEDAC 978-84-18983-00-9
Reservados todos los derechos. Queda rigurosamente prohibida, sin la autorización escrita de los titulares del copyright, bajo las sanciones establecidas en las leyes, la reproducción, transmisión y distribución total o parcial de esta obra por cualquier medio o procedimiento, incluidos la reprografía y el tratamiento informático.

OBJETIVOS

Conocer los diferentes tipos de pruebas software.

Conocer metodologías para la aplicación de pruebas.

Entender el concepto de caso de prueba.

Comprender la relación entre pruebas y calidad.

/ 1. Introducción y contextualización práctica

Actualmente, uno de los factores más determinantes en el *software* es la calidad que podamos ofrecer a nuestros usuarios. En un mercado en el que cada vez existe un mayor número de aplicaciones, es muy complicado que nuestra aplicación pueda destacar.

Por ejemplo, en el desarrollo de aplicaciones en entornos móviles, se observa que aquellas aplicaciones que poseen bloqueos llamados ANR *(Application Non Response)* son desinstaladas directamente por los usuarios. Además, Google o Apple pueden tomar medidas para quitar estas aplicaciones de la tienda correspondiente.

Por esta razón, todos los IDEs de desarrollo actuales incorporan grandes funcionalidades para poder ejecutar pruebas que den cobertura a la mayor cantidad posible del código que hayamos implementado. Se suelen contemplar pruebas para comprobar la funcionalidad y otras pruebas que permiten analizar el rendimiento de la aplicación.



Fig. 1. Las pruebas software requieren de un buen conocimiento de la tecnología.

Si volvemos de nuevo al contexto de aplicaciones móviles, vemos que un factor de calidad puede ser el uso que hagamos de la batería. Por eso, en estos entornos, también se permite observar este parámetro. En este tema, nos centraremos en observar los diferentes tipos de pruebas que existen para poder vincularlos con una mayor calidad en el software que realicemos. Escucha el siguiente audio en el que planteamos el caso práctico que iremos resolviendo a lo largo de esta unidad.

Audio intro. "Realización de pruebas de integración"
https://bit.ly/39z0gl6

/ 2. Planificación de las pruebas

En cualquier desarrollo de una aplicación, es necesario tener en cuenta la calidad del producto que estamos generando. Normalmente, utilizamos una metodología de desarrollo que suele requerir de un proceso para validar las funcionalidades que hemos implementado. Para ello, se llevan a cabo diferentes tipos de pruebas que puedan **garantizar que las funcionalidades que hemos implementado se comporten realmente como se han especificado en las etapas previas de análisis y diseño.**

Esto es importante porque estamos implementando una funcionalidad que está directamente vinculada con un caso de uso requerido por los usuarios. Además, será nuestro cliente final el que diga si la funcionalidad implementada satisface las necesidades de su negocio o aplicación. Por ello, es importante que cuando utilicemos cualquier metodología de desarrollo *software*, exista una **planificación previa** de las pruebas que vamos a tener que pasar, pues se trata de tiempo que vamos a invertir en el desarrollo de la aplicación.

Podemos observar que el tiempo para realizar pruebas es muy importante, aunque no añadan una funcionalidad directa. Las pruebas que realicemos en nuestro sistema están directamente relacionadas con la calidad del software que vayamos a implementar. Nótese que **el tiempo para probar una aplicación debe contemplar tanto el tiempo de realización de la prueba como de comprobar que el software satisface dicha prueba.**

- **Metodologías basadas en pruebas:** Las pruebas son un aspecto tan importante en el desarrollo *software* que muchas metodologías se basan en ellas para poder avanzar correctamente en el desarrollo.

Existen metodologías como TDD y/o *Scrum* que basan todos sus desarrollos y filosofía en disponer de un conjunto bastante completo de pruebas. Gracias a esto, es posible avanzar a mayor velocidad y evitar errores de regresión.

En el caso de la metodología TDD, las pruebas permiten guiar todo el desarrollo *software* para detectar errores lo más rápidamente posible.



Audio 1. "Planificación de las pruebas"

https://bit.ly/2KYr9EO



/ 3. Tipos de pruebas

Existen diferentes tipos de pruebas que permiten testear el *software* desde diferentes puntos de vista y con diferentes enfoques con el fin de poder detectar el mayor número de errores.

Cuando trabajamos en la ejecución de pruebas, **se dice que una prueba ha sido exitosa si ha encontrado un error.** Es extraño pensar de esta forma, sin embargo, hay que considerar que las pruebas están justamente para detectar errores y evitar que se produzcan en producción.

Por este motivo, podemos diferenciar entre diferentes tipos de pruebas. Podríamos decir que existen dos grandes grupos:

- **Pruebas estáticas:** Son aquellas que se realizan sin tener que ejecutar el código de la aplicación. En general, pueden referirse a los documentos y hacer un análisis previo a la ejecución desde un punto de vista lógico. Como hemos comentado, este tipo de pruebas no tiene en cuenta el resultado real de la ejecución del software.

- **Pruebas dinámicas**: Son aquellas que requieren de la ejecución del código para poder comprobar si este se ejecuta como esperábamos. Existen diferentes tipos de técnicas a la hora de utilizar las pruebas dinámicas, por ejemplo, pruebas de caja negra, pruebas de caja blanca, etc.

Dependiendo del contexto y del tipo de prueba, podremos detectar mayor número de errores y, en caso de no encontrarlos, cubrir mayor parte del código.

Además, podríamos clasificar las pruebas según el objetivo que intentan verificar en cuanto a la implementación:

- **Pruebas funcionales:** Este tipo de pruebas son para comprobar si la funcionalidad que hemos implementado se comporta acorde con los requisitos funcionales que capturamos en la fase de toma de requisitos.

- **Pruebas no funcionales:** Este tipo de pruebas está centrado en comprobar si satisfacemos los diferentes requisitos no funcionales capturados en la fase de captura de requisitos

Vídeo 1. "Pruebas en dispositivos móviles"
https://bit.ly/3lvYiUX

/ 4. Caso práctico 1: "Comprobación de funcionalidades"

Planteamiento: En nuestro equipo, se plantea la necesidad de realizar la comprobación de la implementación de todas las funcionalidades capturadas durante nuestras reuniones con el cliente en las fases iniciales del proyecto.

Nudo: ¿Qué tipo de pruebas crees que serían las más adecuadas en esta etapa?

Desenlace: Acorde con lo expuesto en el caso, sería necesario poder trazar un plan de prueba que permita capturar el mayor número de errores posible en la menor cantidad de tiempo para poder comprobar la funcionalidad de cualquier aplicativo. Como desarrolladores *software*, es necesario tener en cuenta estos factores para aumentar la velocidad de desarrollo.

Según nos piden, lo que queremos comprobar es que todas las funcionalidades hayan sido implementadas. Por esto, lo mejor sería realizar **pruebas funcionales**. A través de estas pruebas, podríamos comprobar que todas las funcionalidades capturadas se han cumplido y ver si existe algún fallo de implementación en su desarrollo.

Además, gracias a este tipo de pruebas, podemos observar si existen funcionalidades que no han sido implementadas, lo que puede servir para actualizar la planificación general del proyecto en términos temporales.



Fig. 2. El diseño de pruebas debe tener en cuenta diferentes enfoques.

/ 5. Casos de prueba

Cuando implementamos un *software*, es necesario cumplir con las necesidades establecidas en los requisitos, es decir, en los casos de uso tomados en la fase de análisis.

Decimos que un caso de prueba trata de comprobar que se cumple una funcionalidad determinada de la aplicación bajo unas ciertas condiciones. Para cada uno de los casos de prueba, es necesario **definir el contexto en el que se ejecuta, la acción que se lleva a cabo, el resultado obtenido y el resultado esperado.** Si en este contexto, **el resultado obtenido y el resultado deseado son exactamente el mismo**, se dice que **la prueba se ha pasado correctamente**. En caso contrario, se detecta un error en la solución y, por lo tanto, hay un posible fallo en la implementación.

En muchas ocasiones, detectar el error puede ser también una tarea de **depurar las pruebas** que hemos generado, pues **también pueden existir errores en las pruebas**.

Esto quiere decir que al tratarse de pruebas desarrolladas con código, pueden tener errores en su implementación y dar falsos positivos y falsos negativos. Cuando tenemos en cuenta los casos de uso, puede ser necesario realizar diferentes pruebas por casos de uso.

Una típica pregunta que nos surge es cómo probar la funcionalidad de todo el sistema intentando minimizar el número de pruebas.

Como hemos visto anteriormente, las pruebas pueden requerir de tiempo de desarrollo y recursos. Por ello, **si somos capaces de probar mayores casos de uso o funcionalidades en una única prueba, estaremos adelantando el trabajo.** Sin embargo, este enfoque puede ocultar diferentes errores, ya que puede no tratarse todos los casos de error que pueden existir.



Fig. 3. Las pruebas de usabilidad se tienen muy en cuenta actualmente.

/ 6. Procedimientos en realización de pruebas

Uno de los tipos más importantes de pruebas que nos podemos encontrar son las **pruebas funcionales**. En ellas hay que comprobar que se satisfacen todos los requisitos funcionales capturados en fases anteriores.

Algunos tipos de pruebas funcionales son las siguientes:

- **Pruebas unitarias:** Una prueba es unitaria cuando trata de comprobar el correcto funcionamiento de una determinada unidad de código que tenemos implementado.

  Si una prueba unitaria intenta comprobar el funcionamiento de más de una funcionalidad estaríamos diciendo que la prueba no es unitaria.

  Generalmente, se tiende a pensar que si tenemos todas las pruebas unitarias del sistema y todas se pasan satisfactoriamente de forma individual, el software está libre de errores. Sin embargo, dadas las características del software, esto no sucede de esta forma porque debemos tener en cuenta el contexto de ejecución, entre otros factores.

- **Pruebas de integración:** Son aquellas que se llevan acabo cuando hayan pasado las pruebas unitarias.

  En este caso, tenemos que comprobar que el resto de los elementos se pueden integrar correctamente realizando pruebas en grupo de diferentes tipos de funcionalidades.

- **Pruebas de componentes:** Permiten comprobar que los componentes con múltiples funcionalidades que hemos desarrollado satisfacen las necesidades y se comportan correctamente.

- **Pruebas de regresión:** Normalmente, cuando solucionamos un error, podemos generar otro de manera colateral. Estas pruebas intentan comprobar que no se vuelven a producir errores que anteriormente se hayan solucionado. Es un tipo de pruebas muy importante para determinadas metodologías y, generalmente, suelen estar automatizadas.

Vídeo 2. "Pruebas de integración"

https://bit.ly/36rU0tg

/ 7. Caso práctico 2: "Elección adecuada del tipo de prueba"

Planteamiento: Nuestro equipo de desarrollo acaba de preparar todas las pruebas funcionales que se pasarán al equipo de calidad para que estos testeen el desarrollo de una aplicación que se acaba de implementar.

Las pruebas que hemos desarrollado son pruebas unitarias y confiamos en que cuando pasen, el software esté completamente depurado y no tengamos que realizar ninguna prueba más.

Nudo: ¿Qué opinas de esta situación?

Desenlace: Como hemos visto en el tema, las pruebas unitarias son un tipo especial de pruebas funcionales que tratan de comprobar el flujo de ejecución de nuestra aplicación probando determinados casos de manera independiente.

Con anterioridad, se ha comentado que un típico ejemplo de error sobre las pruebas unitarias es asumir que cuando estas son ejecutadas y pasadas satisfactoriamente, el código va a estar libre de errores, ya que no sabemos qué tipo de dependencias tenemos en nuestra aplicación para poder asegurar que no se van a producir más errores tan solo ejecutando pruebas unitarias.

Es un tipo de prueba muy potente que permiten verificar una funcionalidad muy determinada, sin embargo, no son capaces de comprobar cómo se comportarían estos componentes cuando interactúan con otros de nuestro sistema. Para ello, habría que realizar otro tipo de pruebas como las de integración.

No podemos asegurar que cuando pasemos las pruebas unitarias nuestra aplicación esté libre de errores porque puede que tengamos dependencias con otros componentes y tengamos que realizar la comprobación trabajando junto con estos componentes.

Además, desde el punto de vista de las pruebas, nunca podemos asegurar que nuestro *software* va a estar libre de errores, pues, en ocasiones, escapa a nuestro control. Lo que sí podemos asegurar es que los errores no se han podido reproducir en las condiciones que habíamos probado. Desde el punto de vista de las pruebas, hablamos siempre de reproducción de errores.

Por ello, cuando un cliente nos dice que ha detectado un error, nosotros tenemos que preguntar en qué contexto, bajo qué circunstancias... para poder reproducirlo.

/ 8. Resumen y resolución del caso práctico de la unidad

En este tema, hemos presentado la relación existente entre las **pruebas *software* y la calidad** que podamos ofrecer en el mercado. Como hemos visto, existen **diferentes tipos de pruebas** que tratan de medir diferentes parámetros en el diseño de las aplicaciones. Es necesario que tengamos en mente el tiempo requerido para la realización de pruebas cuando realicemos la **planificación temporal** para el desarrollo de cualquier componente *software*.

Las pruebas son un elemento muy importante que nos permiten guiar todo el desarrollo y evitar posibles errores en el futuro.

Un ejemplo de este tipo de errores, como hemos visto, puede ser los **errores de regresión,** pues es muy común realizar el cambio en un componente que genere un error en otro.

Con las **pruebas de regresión**, nos aseguramos de que el *software* que hemos implementado no va a generar errores por haber arreglado un problema en el presente.

Resolución del caso práctico inicial

A la hora de desarrollar un diagrama de tiempos, es necesario tener en cuenta el tipo de diagrama que queremos desarrollar.

En el enunciado, se nos indica que creemos un diagrama de tiempos para una planificación temporal. En este caso, al tratarse de una planificación de un proyecto, es necesario tener en cuenta los diferentes imprevistos que pueden suceder en el proyecto.

Normalmente, cuando realizamos la planificación de un proyecto, tenemos que considerar los tiempos de margen por si ocurren eventos que pueden retrasarlo. Por ejemplo, parte del equipo se puede enfermar. Estos tiempos son conocidos como **holguras.**

En un diagrama de Gantt, las holguras no se pueden indicar fácilmente, por lo tanto, no es un diagrama adecuado para este caso. En esta ocasión, lo mejor sería utilizar el método PERT, ya que permite introducir estos tiempos.



Fig. 4. Los componentes deben trabajar en conjunto.

/ 9. Bibliografía

Kaner, C.; Bach, J., y Pettichord, B. (2002). *Lessons learned in software testing: a context-driven approach*. New York: Wiley.

Laboon, B. (2017). *Friendly introduction to software testing*. United States: Createspace.